

Supporting Decisions in a Near-Unpredictable Game

A technical description of ABRA, a decision-support model family for competitive Pokémon

Version 3.30.0 · Last updated 2026-07-31 Will Hooper · ABRA

This is a living document, updated in the same pass as any change to the code, together with the deck and the technical documentation. A prior conclusion is never silently rewritten; new information is added and what changed is stated. See `CHANGELOG.md`.

Abstract

ABRA is a decision-support model family for **Pokémon Champions VGC, Regulation M-B, best-of-one closed-sheet ladder**. It continuously ingests public battle replays from Pokémon Showdown and stores the durable facts of every game, then builds small, CPU-trainable models on that store. Its central empirical finding governs its design: **predicting the winner of a game from the two team sheets is near-impossible in this format — even a player-Elo model ties a coin**. ABRA therefore does not sell outcome prediction. It follows the recipe that worked in poker, Diplomacy, and sports analytics: *support decisions, don't predict outcomes*, and judge every model by a proper score against an honest baseline with a confidence interval. This paper states the empirical ceiling, the data model, each model with its validated result (including two honest negatives), the mathematics, the limits, and the road to the in-battle engine (ALAKAZAM).

1. The empirical ceiling (why the design is what it is)

On 600+ held-out real Champions games, a Bradley-Terry player-Elo model reaches a held-out log-loss of **0.687 against a coin's 0.693** — a real but negligible edge. **A 2026-07-25 re-measurement makes the ceiling lower still:** the previously published "higher-rated player wins 55.0%" was computed with a name-only bot filter that missed six high-volume accounts. Removing them gives **52.4%, 95% CI [49.9, 54.9]** — an interval containing a coin flip. A cloned-policy rollout engine (MEDICHAM) does *worse* than a coin as a raw win-predictor (log-loss ≈ 1.2 ; it picks the actual winner on $\sim 44\%$ of decisive calls, i.e. it is systematically *inverted*, over-backing fast offensive teams that lose more). After held-out Platt recalibration it only edges the coin (0.6897 vs 0.6931).

The conclusion is not "our models are weak." It is a property of the game: a two-player, zero-sum, **imperfect-information, simultaneous-move** game with a non-transitive metagame has an irreducible outcome-prediction ceiling from team sheets alone. This is the same reason expected-goals (xG) models in football predict *shot quality* rather than final scores. **Design consequence: stop predicting outcomes; support decisions.** Everything below serves that.

2. Data: store raw, analyse on top

ABRA reads Showdown's public replay API (`search.json?format=` , `search.json?user=` , `<id>.log`); it reads nothing private and creates no accounts (`SECURITY.md`). The extractor (`engine/durable-ingest.js` , `extract()`) turns one battle log into one durable record:

Field	Meaning
id , date	replay id and upload time
p1 , p2	{name, rating, bot} per player
six.p1/p2	the revealed team of six
brought.p1/p2	the four actually brought
lead.p1/p2	the two led
sets	per species, the moves / item / ability the replay <i>revealed</i>
turns	per-turn events (moves, damage, faints, status, field)
winner	the winning name

The store is append-only JSON Lines keyed by replay id: idempotent, deduplicated at read time, and grown hourly by a GitHub Action. The **governing rule** is *store raw, analyse on top*: every filter (rating tier, humans-only, archetype, playstyle) is a re-computation over the store, never a re-pull. Changing how we segment games is free; the fetch is a one-time cost. About 2,600 public games/day are available, and the store grows ~18k/week, so every model below sharpens on its own over time.

3. The validated foundation — exact damage (MEDICHAM)

The one component that is *not* a coin flip is the damage engine. MEDICHAM's Gen-9 doubles damage pipeline (`engine/medicham2-browser.js`) is validated against the Smogon damage calculator (the community ground-truth) on 31 meta scenarios: **within 5% on 100% of scenarios, median error 0%, worst 3%** (16-roll rounding). This is gated in CI (`engine/validate_damage.js` → `data/damage-validation.json`). Every model that reasons about damage builds on this, and "will this move KO?" is a *winnable* prediction, unlike "who wins the game."

4. The models and their validated results

Every probability ships a **proper score** (log-loss and/or Brier), a **confidence interval** (clustered by game where states within a game are correlated), and an **honest baseline**, persisted to JSON and gated in CI.

4.1 GURU — meta / matchup matrix (descriptive)

From REAL outcomes, `engine/guru.py` builds an archetype × archetype matchup matrix (K is chosen from the data; see `data/archetypes.json`) over the generated game count in `data/live.js` (hardcoded sizes are retracted, S13) — games, each cell a win-rate with a **Wilson score interval**. GURU is *descriptive*: its own predictive test shows per-game winner prediction from the matrix ties a coin (log-loss 0.7122 vs 0.6931), exactly as §1 predicts. Its value is honest matchup structure with error bars, and it is the real (not simulated) payoff matrix that SLOWKING solves. Output: `data/guru-matchups.json` , `data/guru.js` .

4.2 XATU — opponent belief (modest, useful)

`engine/xatu.py` learns, per species, the set (item/ability/moves) usually run, and predicts the opponent's next move from state. On held-out human moves the behaviour-clone reaches **top-1 35.9% (CI 35.2–36.5), top-3 71.6%**, cross-entropy 2.27 nats — beating a species-agnostic baseline (4.54) and uniform-over-moveset (2.91). A modest but real signal; human move choice has genuine entropy. Output: `data/xatu.json` , `data/xatu.js` ; harness `engine/eval_policy.py` → `data/policy-eval.json` .

4.3 PORY — mid-game win probability (RETRACTED as a value net; it is material arithmetic)

The pivot's proof. `engine/pory.py` reconstructs per-turn board state (mons alive out of four, mean active HP, turn) and fits a logistic value net. Held-out, clustered by game: **log-loss 0.567 vs coin 0.693**, beating a material-sign heuristic, **calibrated to ECE 1.6%, CI [0.548, 0.583]**. The *live board is predictable even though the pre-game sheets are not* — the thesis, demonstrated. PORY is wired into KADABRA as a per-turn "you're at X%". Output: `data/pory.js` ; report `data/pory-eval.json` .

4.4 CHOMP-EV — do CHOMP's brings beat humans'? (honest NULL)

The winnable team-preview test. For each held-out game (both full sixes, both actual brings, the winner), `engine/chomp_ev.js` ranks each side's *actual* bring among all 15 candidate brings by CHOMP's exact-damage coverage, and asks whether that quality signal tracks who won. On **1,205 games**: CHOMP's bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), ties an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). It is **robust to forfeits** (0.505; a forfeit is usually a concession from a losing position, and dropping all forfeits does not change the result), and a measured **selection audit** shows the required "all four revealed" filter is a mild bias (eval 6.5 turns / 1280 rating vs 6.08 / 1267 excluded) that, if anything, *favours* CHOMP — making the null conservative. A **belief-weighted** variant (coverage vs the opponent's likely-4) also ties the coin (0.6924). Interpretation: the bring decision sits at the same near-coin ceiling as pre-game prediction; CHOMP's damage math stays validated and useful as a calculator, but "CHOMP builds better brings" is not yet empirically supported. This negative is a guardrail: it stops optimising a bring metric that carries no held-out winning signal . Report `data/chomp-ev.json` ; test `tests/test-chomp-ev.js` .

A phrasing the filter itself mandates. `require_full_bring` conditions on game length: measured 2026-07-31, the games it keeps are **1.71x longer** on average (7.4 vs 4.3 mean turns; 19,589 kept vs 8,713 dropped). Every bring statistic in this project is therefore "*the bring, among games long enough to show it*", which is not the same as "the bring". `data/quality-filter.json` states this at the point of filtering and requires it to be said downstream; this is that.

4.5 SLOWKING — team-preview Nash and the playstyle cycle (suggestive)

`engine/slowking_preview.py` solves a matchup matrix to a mixed-strategy equilibrium and grades it by **exploitability** (the worst-case win-edge a best pure counter extracts; lower is better; Nash $\approx$ 0), against greedy "single best deck" and uniform baselines, with a bootstrap CI that propagates matchup-count uncertainty (Beta resampling). Over GURU's 13 species-archetypes the equilibrium is far less exploitable than uniform (Nash $\approx$ 0 vs 0.109), but greedy $\approx$ Nash because this meta is currently near-transitive (a dominant deck). A **playstyle** re-analysis (`engine/playstyle.js` classifies each team as TrickRoom / Rain / Sun / Sand / Snow / Setup / PerishTrap / TailwindOffense / FakeOutBalance / Stall / HyperOffense) surfaces a non-transitive cycle — **TrickRoom** → **HyperOffense** → **Sand** → **TrickRoom** — with a point exploitability gap of $\sim$ 0.073 for greedy; the equilibrium now correctly leads with Sun ($\sim$ 31%), since Reg M-B Charizard is Mega-Y (Drought) and is classified as a Sun setter. **Honest caveat:** each cycle leg rests on only 13–18 games (win rates 73% / 71% / 67%) with 95% CIs that cross 50%, so the cycle is a **suggestive pattern, not a settled fact**; it will sharpen as the store grows. Where matchups *are* well-sampled they tend to run flat against intuition — **Rain vs Sun is 51% (n=236)** and **Tailwind vs no-Tailwind is 47% (n=756)**, both statistical coin-flips. Reports `data/slowking-eval.json` , `data/slowking-playstyle-eval.json` ; test `tests/test-slowking.py` .

5. Mathematics

Wilson score interval (used for every matchup rate) for w wins in n games, $z = 1.96$: $(\hat{p} + z^2/2n \pm z \cdot \sqrt{(\hat{p}(1-\hat{p})/n + z^2/4n^2)}) / (1 + z^2/n)$, with $\hat{p} = w/n$. It is well-behaved at small n and near 0/1, unlike the normal approximation.

Value net. Features $x = [\text{alive_diff}, \text{hp_diff}, \text{my_alive}, \text{foe_alive}, \text{turn}/10]$ are standardised by train mean/std; $P(\text{win}) = \sigma(w \cdot z + b)$. Graded by held-out **log-loss** $-(y \cdot \ln p + (1-y) \cdot \ln(1-p))$ and **Brier** $(p-y)^2$; the coin scores $\ln 2 = 0.6931$ and 0.25 respectively.

Discrete choice — the scoring bot's policy (v3.28.0). A player facing a turn chooses one of the legal (move, target) pairs. Writing x_j for the attributes of alternative j — type effectiveness against that specific target, base power, whether the move is already dead on the board, and the behaviour clone's $P(\text{move} \mid \text{species})$ — the conditional logit model (McFadden 1974) is

$$P(\text{pick } j) = \exp(w \cdot x_j) / \sum_k \exp(w \cdot x_k),$$

with w estimated by maximising $\sum_i [w \cdot x_{\{i, \text{chosen}\}} - \ln \sum_k \exp(w \cdot x_{\{i, k\}})]$ over **146,910** real human decisions from **6,091** clean open-sheet games (117,824 train / 29,086 held out), with a feature vector that has grown from 12 to **53**. The weights are **estimated, never written down**, and the realism report is never consulted during fitting — it is held back as the out-of-sample check, because a diagnostic stops being evidence once it becomes the objective.

Held out **by game** (decisions within a game are correlated — the same clustering argument as the CIs below): logL/decision **-1.6006** and top-1 **33.6%**, against the behaviour clone alone at $-1.9302 / 27.1\%$ and uniform at $-1.7627 / 24.1\%$. Open-sheet games are used because they are the only corpus in which the **choice set** is known rather than guessed: a normal replay reveals only the moves that were *used*, so alternatives reconstructed from revelation are biased by revelation.

The model's known limitation is **independence of irrelevant alternatives**: logit implies the odds between two options are unaffected by what else is on the menu, which fails for close substitutes (the red-bus/blue-bus problem). A set carrying two moves of the same type is exactly that case. Nested or mixed logit is the remedy and neither is implemented, so the fitted probabilities are a good ranking and only an approximate distribution.

Equilibrium and exploitability. Each preview is a two-player zero-sum matrix game on an antisymmetric edge matrix $M[i, j] = (p(i > j) - p(j > i))/2$. Regret matching (Hart & Mas-Colell) converges to an ϵ -Nash. For a strategy x , **exploitability** $= -\min_j (x \cdot M[:, j])$ — the worst-case loss to a best response; the Nash value is 0, so a Nash strategy scores ≈ 0 and a predictable single-deck strategy is punished.

Confidence intervals. Because per-turn states within a game are correlated, CIs are **bootstrapped by resampling games** (clustered), not states. Matchup-matrix uncertainty is propagated by **Beta($n \cdot p + 1$, $n \cdot (1-p) + 1$) resampling** of each cell before re-solving.

Future rating math. For any descriptive meta-rating we will use an **intransitivity-capable** class (blade-chest / low-rank bilinear, Chen & Joachims 2016; or Nash-averaging, Balduzzi et al. 2018) and a **Helmholtz–Hodge / HodgeRank** decomposition (Jiang, Lim, Yao & Ye 2011) to split the matchup flow into a transitive ranking plus a cyclic (rock-paper-scissors) component — the correct tool for "which cores beat which" and for quantifying how cyclic the meta really is.

6. Limitations and honest ceilings

1. **The game-winner ceiling is permanent** (Elo $\approx$ coin). SLOWKING/ALAKAZAM are judged on decision quality and self-play/ladder win-rate, never on match-outcome prediction.

2. **Revealed sets are partial** (a mon that never attacked reveals no moves); belief is a lower bound.
3. **Small samples in the meta layer.** Playstyle and core matchups are thin; those results are suggestive until the store grows.
4. **Policy is the residual GIGO — and in 3.28.0 the binding constraint is the OBJECTIVE, not the knowledge.** The damage is validated. The policy now runs a real damage calculation and does decide switches — both were listed here as missing and both became false, and they are corrected rather than quietly dropped. What remains: **one ply**, no model of the opponent's move, no search.

The sharper limitation is measured. Over 2026-07-30, **four separate feature additions produced four nulls**, while **two changes to the objective produced two large wins** — greedy action selection at +12 points (79.7% of decisive pairs) and self-play policy improvement at 55.9%. An overdispersion check across teams (~1.00, against 1.169 for a known real effect) rules out the obvious confound, so the nulls are genuine. Adding knowledge to an imitation-fitted policy has stopped paying.

RECONCILED 2026-07-31. *That 55.9% was measured on the 53-feature vector with switching OFF. Repeating the experiment on the 56-feature vector with switching ON gives 48.1% [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

The cleanest demonstration is the pair-scoring layer (DODUO), which is **built, wired, controlled and measured, and loses at 42.0%** [39.9, 44.3] over 1,934 seed-paired games against its own zeroed control. Its fit prices "use a spread move beside my own ally that does not hurt it" at **-5.054** — a statement that humans rarely click it, not that it is bad. Refitting those weights for *winning* rather than *resemblance* is untested and is the project's top open question.

A separate class of defect, worth naming because it is not a modelling disagreement: a fact reaching one consumer and not the next. Priority blocking lived in the tag artifact and never reached the simulator, so **Sucker Punch beat a Farigiraf in every rollout ever run.** A switch-in's own ability never reached the code that chooses the switch — over 40,001 matchups, declaring Intimidate, Drizzle or Drought moved the feature vector in **zero** of them against a control's 2,754. And every mega forme carried a null ability, an empty moveset and no item, so **26.0% of the format's usage scored as threatening nothing.** All fixed; a gate (`engine/artifact_audit.js`) now compares derived artifacts against their sources, because nothing had.

Every `build_lab` win rate on record was measured against the older board-blind pilot and none has been re-run, so all of them remain provisional.

1. **Champions rule specifics** (sleep/paralysis edge cases) are flagged, not yet fully modelled.

7. The road to ALAKAZAM

ALAKAZAM is the in-battle capstone, built last on the inputs above. Given a live position it will output the win-%-optimal move (a mixed strategy) and its value by: (1) a **belief** over the opponent's hidden sets (XATU), updated by a Bayesian filter; (2) **depth-limited search** over the validated damage engine, solving each simultaneous turn as a **matrix game** (regret matching — this removes the speed bias that inverted the greedy engine); (3) a **learned value** at the leaves (PORY, grown to an NNUE-style net); (4) **human-anchoring** (KL-regularised to the behaviour-clone) so it stays strong and unexploitable. Inference is light (CPU / Web Worker / WASM); the strongest version needs offline RL on millions of human + self-play games

and a rented cloud GPU. It is judged on decision quality and self-play/ladder win-rate with CIs — never on predicting the winner. A self-play data engine (MEW) is the pacing item toward the millions of games that path needs.

8. References

1. Zinkevich et al., *Regret Minimization in Games with Incomplete Information* (CFR), 2007.
 2. Lanctot et al., *Monte Carlo Sampling for Regret Minimization* (MCCFR), 2009.
 3. Moravčík et al., *DeepStack*, Science 2017. · Brown & Sandholm, *Libratus*, Science 2018.
 4. Brown et al., *Combining Deep RL and Search* (ReBeL), NeurIPS 2020. · Schmid et al., *Player of Games*, 2021.
 5. Angliss et al., *VGC-Bench*, arXiv 2506.10326, 2025. · UT-Austin-RPL, *Metamon* (offline RL + transformers), arXiv 2504.04395, 2025.
 6. Perolat et al., *DeepNash / R-NaD* (Stratego), Science 2022. · Vinyals et al., *AlphaStar*, 2019.
 7. Meta FAIR, *CICERO / piKL* (human-regularised RL, Diplomacy), Science 2022.
 8. Chen & Joachims, *Modeling Intransitivity in Matchup Data* (blade-chest), WSDM 2016. · Balduzzi et al., *Re-evaluating Evaluation* (Nash-averaging), NeurIPS 2018.
 9. Jiang, Lim, Yao & Ye, *Statistical Ranking and Combinatorial Hodge Theory* (HodgeRank), 2011.
 10. Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*, JASA 1927.
 11. McFadden, *Conditional Logit Analysis of Qualitative Choice Behavior*, in Zarembka (ed.), **Frontiers in Econometrics**, Academic Press 1974 — the discrete-choice model the scoring bot's policy is fitted with (§5).
 12. the Smogon damage calculator — community damage ground-truth. · Pokémon Showdown replay API.
-

Companion documents. [Slide deck](#) · [Technical documentation](#) · [Model ledger](#) · [Changelog](#)

The role family: multi-label composition, WAR, and emergent roles (v2.6.0)

Motivation

The earlier playstyle model assigned each team exactly one archetype. This is a **multi-class** framing of a **multi-label** object: a real team is Sun *and* Tailwind *and* Fake Out at once. Forcing one label discards most of the information and shatters the data into archetype×archetype cells of $n \approx 11-18$, which is why those matchup numbers were untrustworthy. The literature is explicit: multi-label classification (Tsoumakas & Katakis 2007), team-as-mixture-of-latent-roles (topic models; Blei-Ng-Jordan 2003), and latent roles beating raw identity for outcome prediction in team sports (arXiv 2304.08272).

Role tagging (leak-free, data-earned)

We define 26 functional roles. A **species earns a role from data** — it is credited once it is observed performing the role (≥ 2 times) across the store. Multi-effect moves carry several *factual* roles (Matcha Gotcha = special+heal+status; Body Press = wall+attack; Fake Out = tempo, not attacker). Role *presence* is binary; graded *strength* is deliberately **not** hand-set (asserting weights violates the project's measurement standard). A team's role vector is built from the **team-preview six**, which are public in every closed-sheet game, so the representation is uncensored and non-leaking.

Each ordered role pair (a, b) aggregates outcomes across every game where one side had a and the other had b, with a Wilson score interval. Because roles co-occur, each game contributes to many cells, so the **median cell rises from $n \approx 15$ to $n = 20$ across 1,051 cells (measured 2026-07-25 on 1,061 quality-filtered games)**. The figure of 7,971 published in v2.6.0 was retracted in 2.7.0 as an artifact of over-tagging (19.6 of 26 roles per team); it has since gone $7,971 \rightarrow 95 \rightarrow \sim 50 \rightarrow 20$ as the taxonomy sharpened and the games were filtered — the structural fix. Empirically, however, a logistic model on the preview role-difference vector predicts the winner at held-out log-loss 0.694 vs a coin's 0.693: **roles describe and attribute, they do not predict**. The per-role coefficients are read as **win-credit per role**; KO-credit per species is measured directly from the turn log.

WAR — Wins Above Replacement (species RAPM)

To attribute wins to individual Pokémon while controlling for teammates and opponents, we use basketball's **Regularized Adjusted Plus-Minus**. With one row per game, label $y = 1$ if p1 won and features $x_s = 1[s \in p1 \text{ six}] - 1[s \in p2 \text{ six}]$, a ridge logistic regression yields β_s , species s's adjusted win contribution. Ridge shrinks rare species toward zero. With replacement β at the 20th percentile and the logistic slope $1/4$ at $p = 0.5$,

$$\text{WAR}_s = 0.25 \cdot (\beta_s - \beta_{\text{replacement}}) \cdot (\text{games } s \text{ appeared}).$$

Held-out, the species model reached log-loss 0.6875 against a coin's 0.6931 — a result now **withdrawn 2026-07-25** — that figure was measured on the UNFILTERED store; on quality-filtered games WAR scores 0.7048 against a coin's 0.6931 (accuracy 0.502). The apparent signal was four bot accounts playing one team in 1,446 games. It does not beat a coin: *which specific species* you bring at preview carries a small real signal that roles and raw sheets do not. Leaders are Basculegion, Kingambit, Sylveon; trailers are negative. Effect sizes are small and magnitudes ridge-shrunk — reported as an exploratory ordering, not settled wins.

Emergent roles by NMF

Rather than hand-declaring roles, we factorize the data with **Non-negative Matrix Factorization** (Lee & Seung 1999): $X \approx WH$ with $W, H \geq 0$, so each team is a non-negative **blend** of latent roles and each role is a recipe over features. Two cuts: (1) the team×move usage matrix (usage-weighted, which down-weights the closed-sheet censoring skew) recovers **offensive cores** but is dominated by attacking moves (relative reconstruction error 0.79); (2) the team×role matrix recovers **six clean archetypes** (error 0.53):

Intimidate+Fake-Out control, physical offense, special offense+sustain, bulky wall+screens+ redirection, Tailwind+Encore, priority. A move's loading on a role is **learned, not typed** — this is the principled source of graded primary/secondary strength (Label Distribution Learning, Geng 2016). The rank and the human names are the only non-data choices. Reconstruction error is **not** comparable across weightings; the correct model-selection criterion is **topic coherence** (Mimno et al. 2011), noted as the next refinement.

Honest limits

Preview-composition signal is small; role-level winner-prediction ties a coin and WAR barely clears it. Role tags are a censored lower bound on capability (closed sheets reveal only used moves). NMF factors are soft and attacker-dominated at the move level. None of these is hidden; each is reported with its baseline.